

Punteros, Referencias y Modelo de Memoria

Semana 7 — Modelo de memoria de un programa, identidad de objetos, mutabilidad, aliasing, paso de argumentos, y gestión de memoria en el sistema operativo

1. ¿Por qué estudiar el modelo de memoria?

Hasta la Semana 6 hemos escrito programas que crean variables, invocan funciones y manipulan listas, pero nunca nos preguntamos *dónde* viven esos datos ni *cómo* el sistema los organiza en la memoria física. Esa pregunta no es un capricho teórico: la comprensión del modelo de memoria es la clave para explicar por qué ciertos comportamientos de Python que parecen misteriosos —como que modificar una lista dentro de una función «se vea» fuera de ella, mientras que reasignar un entero no— son, en realidad, consecuencias directas de cómo el lenguaje gestiona las referencias a los objetos.

Comprender el modelo de memoria permite:

- **Predecir** el comportamiento de las funciones cuando reciben argumentos mutables e inmutables.
- **Diagnosticar** errores sutiles causados por aliasing involuntario (dos variables que apuntan al mismo objeto).
- **Diseñar** funciones con contratos claros sobre efectos secundarios.
- **Conectar** el mundo del lenguaje de alto nivel con la arquitectura de Von Neumann estudiada en la Semana 1 y la pila de ejecución introducida en la Semana 4.

Conexión con semanas anteriores

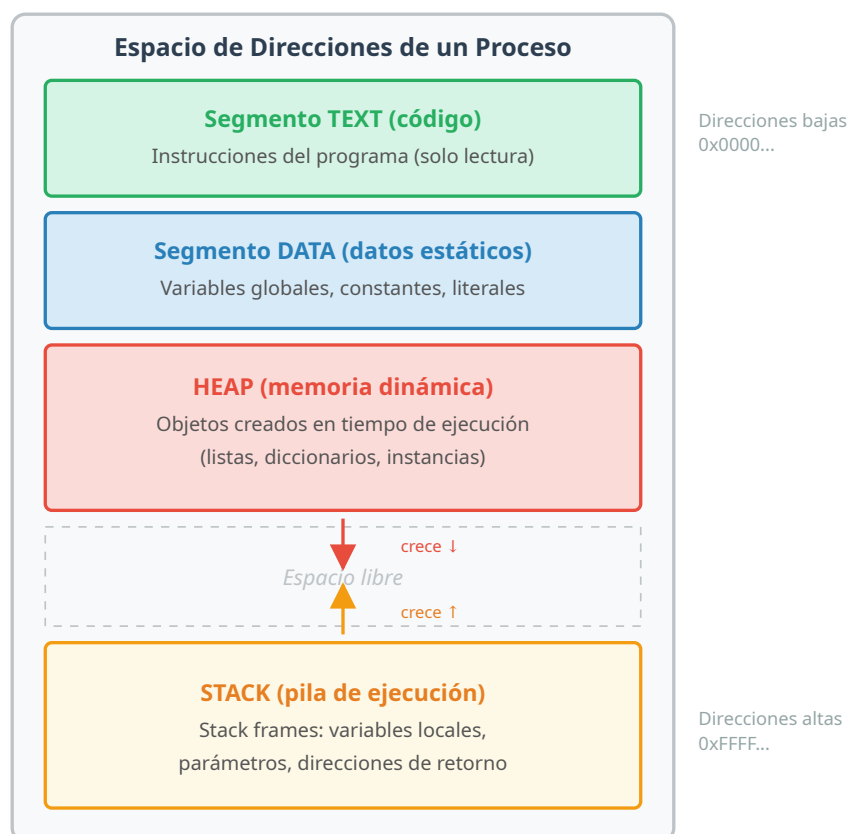
En la **Semana 1** aprendimos que la CPU trabaja con datos almacenados en *memoria*, y que el ciclo fetch-decode-execute depende de direcciones de memoria. En la **Semana 4**, al estudiar funciones, vimos que cada invocación

crea un *stack frame* con sus variables locales. En la **Semana 6**, al implementar algoritmos de ordenamiento *in-place*, realizamos *swaps* que modifican la lista original. Ahora formalizamos todo esto: ¿qué significa exactamente que una variable «apunte a» un dato? ¿Qué pasa cuando dos variables apuntan al mismo objeto? ¿Y cómo interactúa esto con la pila de ejecución del sistema operativo?

2. El Modelo de Memoria de un Programa

Cuando un programa se carga en memoria para ejecutarse, el sistema operativo le asigna un espacio de direcciones virtuales dividido en cuatro regiones principales. Cada región cumple un rol específico, y la comprensión de su función es fundamental para entender cómo se almacenan y acceden los datos durante la ejecución.

2.1. Los cuatro segmentos de memoria



Los cuatro segmentos en detalle

Text (código): contiene las instrucciones de máquina del programa compilado (o, en Python, el bytecode generado por CPython). Es de *solo lectura*: el programa no puede modificar su propio código durante la ejecución. En la Semana 6 vimos que CPython compila el código fuente a bytecode antes de ejecutarlo; ese bytecode reside aquí.

Data (datos estáticos): almacena variables globales y constantes cuya vida útil coincide con la del programa completo. Los *literales* de cadena y los números pequeños que Python almacena en caché (los enteros de -5 a 256) viven en esta región.

Heap (montículo): región de memoria dinámica donde se crean los objetos en tiempo de ejecución. Cada vez que Python ejecuta `[1, 2, 3]`, `{"clave": "valor"}` o cualquier constructor de objeto, el resultado se almacena en el heap. El heap crece «hacia abajo» (hacia direcciones mayores).

Stack (pila): crece en dirección opuesta al heap. Cada invocación de función apila un *stack frame* que contiene los parámetros, las variables locales y la dirección de retorno. Cuando la función termina, su frame se desapila. Revisamos este mecanismo en la Semana 4 al estudiar las instrucciones CALL y RET de la CPU.

Stack vs. Heap: crecimiento en dirección opuesta

El stack y el heap comparten el espacio libre entre ambos y crecen en direcciones opuestas. Si un programa consume demasiado heap (por crear muchos objetos sin liberar) o demasiado stack (por una recursión muy profunda), ambas regiones colisionan. El resultado es un **stack overflow** (desbordamiento de pila) o un error de asignación de memoria (*out of memory*). En Python, la recursión sin caso base produce `RecursionError` antes de que el SO detecte el desbordamiento, porque CPython impone un límite configurable de profundidad (por defecto, 1000 frames).

2.2. ¿Dónde vive cada dato en Python?

Python es un lenguaje donde **todo es un objeto**. Incluso los enteros, los flotantes

y los booleanos son objetos que residen en el heap. Las variables no «contienen» datos: contienen *referencias* (direcciones de memoria) a objetos que viven en el heap. Esta distinción es crucial y diferencia a Python de lenguajes como C, donde una variable entera `int x = 42`; almacena directamente el valor 42 en la posición de memoria asignada a `x`.

```
# ¿Dónde vive cada cosa?
```

```
x = 42          # x es una referencia (en el stack) al objeto i
nombre = "Ada"  # nombre referencia al objeto str "Ada" (en el
notas = [7, 9, 4] # notas referencia al objeto list [7, 9, 4] (er
                  # a su vez, la lista contiene referencias a los

                  # La función id() devuelve la dirección de memc
print(id(x))     # Ejemplo: 140234866713840
print(id(nombre)) # Ejemplo: 140234866501680
print(id(notas))  # Ejemplo: 140234865927040
```

La función `id()`: la «dirección postal» de un objeto

En CPython (la implementación estándar de Python), `id(obj)` devuelve la dirección de memoria donde reside el objeto. Dos objetos distintos siempre tienen `id()` diferentes (mientras ambos existan). Dos variables con el mismo `id()` apuntan al **mismo objeto** en memoria —son *alias*.

3. Referencias, Identidad y el Operador `is`

En Python, la **asignación** no copia datos: crea una nueva referencia al mismo objeto. Esta semántica tiene consecuencias profundas que debemos comprender antes de pasar argumentos a funciones.

3.1. Asignación: crear una referencia, no una copia

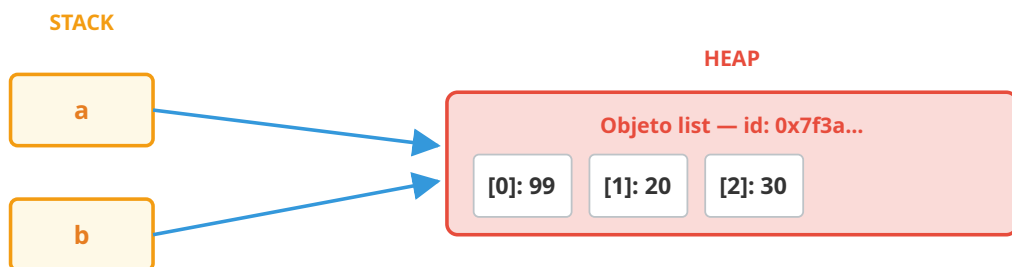
```

a = [10, 20, 30]
b = a                                # b NO es una copia: b apunta al MISMO objeto

print(id(a) == id(b))               # True – misma dirección
print(a is b)                       # True – son el mismo objeto

b[0] = 99                           # Modificamos a través de b...
print(a)                            # [99, 20, 30] – ¡a también cambió!

```



Ambas variables apuntan al mismo objeto: aliasing

3.2. Igualdad vs. identidad: == vs. is

Python distingue dos tipos de comparación:

Operador	Pregunta que responde	Qué compara
==	¿Los objetos tienen el <i>mismo valor</i> ?	Contenido (invoca <code>__eq__</code>)
is	¿Son el <i>mismo objeto</i> en memoria?	Identidad (<code>id(a) == id(b)</code>)

```

# Igualdad vs. Identidad
lista1 = [1, 2, 3]
lista2 = [1, 2, 3]                # Otro objeto con el mismo contenido

```

```

print(lista1 == lista2) # True – mismo valor
print(lista1 is lista2) # False – objetos diferentes

# Caso especial: caché de enteros pequeños
p = 256
q = 256
print(p is q) # True – CPython reutiliza el objeto para valores pequeños

r = 257
s = 257
print(r is s) # False – fuera del rango de caché, se crean nuevos objetos

```

Nunca use `is` para comparar valores

El operador `is` debe usarse exclusivamente para comparar con `None`: `if resultado is None:`. Para comparar valores numéricos, cadenas o listas, use siempre `==`. El comportamiento de `is` con enteros y cadenas depende de optimizaciones internas de CPython que pueden cambiar entre versiones y no forman parte de la especificación del lenguaje.

4. Mutabilidad e Inmutabilidad

Los objetos en Python se clasifican en dos categorías fundamentales según si su contenido puede modificarse después de la creación:

Tipo	Mutable	Inmutable	Consecuencia
int, float, bool		✓	Reasignar crea un objeto nuevo
str		✓	<code>s[0] = 'x'</code> produce <code>TypeError</code>
tuple		✓	No se puede agregar, quitar ni modificar elementos

Tipo	Mutable	Immutable	Consecuencia
list	✓		Se puede modificar <i>in-place</i> ; aliasing visible
dict	✓		Se pueden agregar, quitar y cambiar pares clave-valor
set	✓		Se pueden agregar y quitar elementos

4.1. Inmutabilidad: la reasignación crea un objeto nuevo

Cuando operamos sobre un objeto inmutable, Python no modifica el objeto existente sino que crea uno nuevo y reapunta la variable:

```
x = 10
print(id(x)) # Ejemplo: 140234866712800

x = x + 1    # ¡No modifica el objeto 10!
print(id(x)) # Ejemplo: 140234866712832 – dirección diferente
              # x ahora apunta a un NUEVO objeto int con valor 11
              # El objeto 10 sigue existiendo (si alguien más lo r
```



4.2. Mutabilidad: modificación in-place

Con objetos mutables, las operaciones que modifican el contenido actúan *sobre el mismo objeto*, sin crear uno nuevo:

```
notas = [7, 9, 4]
print(id(notas))    # Ejemplo: 140234865927040

notas.append(10)    # Modifica el MISMO objeto
print(id(notas))    # 140234865927040 — ¡misma dirección!

notas[0] = 8        # También modifica in-place
print(id(notas))    # 140234865927040 — sigue siendo el mismo objeto

# ¿Pero qué pasa si reasignamos?
notas = [1, 2, 3]   # Crea una lista NUEVA
print(id(notas))    # Dirección diferente — notas apunta a otro obj
```

Regla fundamental: modificación vs. reasignación

Modificar un objeto mutable (`lista.append(x)`, `lista[i] = x`, `dict[k] = v`) actúa sobre el mismo objeto — todos los alias ven el cambio.

Reasignar una variable (`variable = nuevo_valor`) crea una nueva referencia, independientemente de si el tipo es mutable o inmutable — los alias anteriores no se afectan.

5. Aliasing: Cuando Dos Variables Apuntan al Mismo Objeto

El **aliasing** (del inglés *alias*: seudónimo) ocurre cuando dos o más variables referencian el mismo objeto en memoria. Con objetos inmutables, el aliasing es inofensivo porque el objeto no puede cambiar. Con objetos **mutables**, el aliasing

es una fuente frecuente de errores sutiles.

5.1. Aliasing inofensivo (inmutables)

```
a = 42
b = a      # Aliasing: a y b apuntan al mismo int 42
b = b + 1  # b ahora apunta a un NUEVO int 43
print(a)   # 42 – a no se afectó
print(b)   # 43
```

5.2. Aliasing peligroso (mutables)

```
def agregar_aprobado(lista_notas, nueva_notas):
    """Agrega una nota a la lista si es aprobatoria (≥ 4)."""
    if nueva_notas >= 4:
        lista_notas.append(nueva_notas)
    return lista_notas

notas_parcial = [7, 9, 5]
resultado = agregar_aprobado(notas_parcial, 8)

print(resultado)           # [7, 9, 5, 8]
print(notas_parcial)       # [7, 9, 5, 8] – ¡también cambi
print(resultado is notas_parcial) # True – son el mismo objeto
```

5.3. Cómo evitar aliasing no deseado: copias

Cuando necesitamos trabajar con una copia independiente de un objeto mutable, debemos crear una copia explícita:

```
# Tres formas de copiar una lista (copia superficial)
original = [10, 20, 30]

copia1 = original[:]          # Slicing completo
copia2 = list(original)       # Constructor list()
copia3 = original.copy()      # Método .copy()

# Las tres son objetos distintos con el
print(copia1 == original)     # True (mismo valor)
print(copia1 is original)     # False (objetos distintos)

copia1[0] = 99
print(original)               # [10, 20, 30] – no se afecta
```

Copia superficial vs. copia profunda

Los tres métodos anteriores crean una **copia superficial** (*shallow copy*): copian las referencias de primer nivel, pero si la lista contiene objetos mutables (como sublistas), esos objetos internos siguen siendo compartidos. Para listas de listas, usaríamos `import copy; copia = copy.deepcopy(original)` para obtener una **copia profunda** (*deep copy*) que duplica recursivamente todos los objetos internos.

```
# Copia superficial con listas anidadas: el peligro
matriz = [[1, 2], [3, 4]]
copia_sup = matriz[:]

copia_sup[0][0] = 99 # Modifica la sublista compartida
print(matriz)        # [[99, 2], [3, 4]] – ¡también cambió!

# Solución: copia profunda
import copy
copia_prof = copy.deepcopy(matriz)
copia_prof[1][1] = 0
```

```
print(matriz)          # [[99, 2], [3, 4]] – no se afecta
```

6. Paso de Argumentos a Funciones

Una de las preguntas más comunes en la enseñanza de Python es: «¿Python pasa argumentos por valor o por referencia?». La respuesta precisa es que Python utiliza un mecanismo llamado **paso por asignación** (*pass by assignment*, también llamado *pass by object reference*): lo que se pasa a la función es una **copia de la referencia**, no una copia del objeto. Esto tiene consecuencias diferentes según el tipo del objeto sea mutable o inmutable.

Mecanismo de paso de argumentos en Python

La clave: se copia la referencia, no el objeto

Cuando invocamos `funcion(argumento)`, Python:

1. Evalúa la expresión `argumento` y obtiene una referencia al objeto.
2. Crea un nuevo nombre (el parámetro formal) en el stack frame de la función.
3. Hace que ese nombre apunte al **mismo objeto** que el argumento.

Dentro de la función, el parámetro es un *alias* del argumento. Lo que suceda depende de si la función **modifica** el objeto o **reasigna** el parámetro:

- **Modificar** el objeto (ej. `param.append(x)`): el cambio es visible fuera, porque ambos nombres apuntan al mismo objeto.
- **Reasignar** el parámetro (ej. `param = nuevo_valor`): solo afecta al nombre local; el argumento original no cambia.

6.1. Con argumentos inmutables: comportamiento «por valor»

```
def intentar_duplicar(n):
    """Intenta duplicar el valor del parámetro.

    Precondición: n es un número.
    Postcondición: devuelve n * 2 (el argumento original NO se modifica)
    """
    print(f"  Dentro (antes):  n = {n}, id = {id(n)}")
    n = n * 2 # Reasignación: n apunta a un nuevo int
    print(f"  Dentro (después): n = {n}, id = {id(n)}")
    return n

# === Sección algorítmica ===
valor = 21
print(f"Antes de llamar:  valor = {valor}, id = {id(valor)}")
resultado = intentar_duplicar(valor)
print(f"Después de llamar: valor = {valor}, id = {id(valor)}")
print(f"Resultado:         {resultado}")
```

Salida esperada

Antes de llamar: valor = 21, id = 140234866713152

Dentro (antes): n = 21, id = 140234866713152 ← mismo id

Dentro (después): n = 42, id = 140234866713824 ← id diferente

Después de llamar: valor = 21, id = 140234866713152 ← sin cambios

Resultado: 42

6.2. Con argumentos mutables: comportamiento «por referencia»

```
def agregar_promedio(notas):
    """Agrega el promedio al final de la lista de notas.
```

Precondición: notas es una lista no vacía de números.

Postcondición: la lista original queda modificada con el promedio.

Efecto secundario: modifica la lista recibida.

"""

```
promedio = sum(notas) / len(notas)
```

```
notas.append(round(promedio, 2)) # Modifica el objeto: visible
```

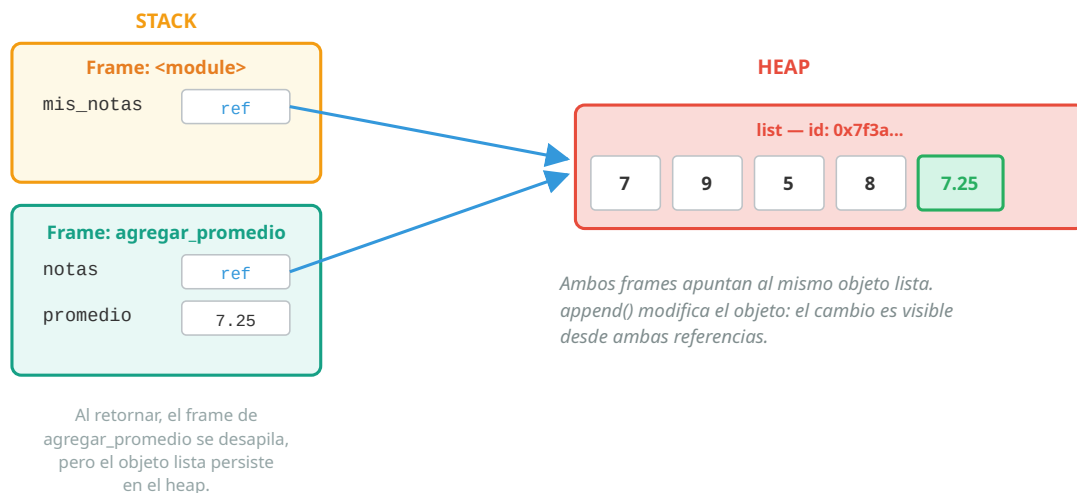
```
# == Sección algorítmica ==
```

```
mis_notas = [7, 9, 5, 8]
```

```
print(f"Antes: {mis_notas}")
```

```
agregar_promedio(mis_notas)
```

```
print(f"Después: {mis_notas}") # [7, 9, 5, 8, 7.25] - ¡modificada
```



6.3. Efectos secundarios: documentar o evitar

Un **efecto secundario** (*side effect*) es cualquier modificación de estado que una función realiza fuera de su ámbito local: modificar un argumento mutable, escribir en un archivo, imprimir en pantalla, o alterar una variable global. El principio de **menor sorpresa** exige que los efectos secundarios sean explícitos en la documentación de la función.

Patrón: función sin efectos secundarios (preferible)

Crear una copia interna del argumento y trabajar sobre ella:

```
def ordenar_sin_modificar(datos):
    """Devuelve una nueva lista con los datos ordenados.

    Precondición: datos es una lista de elementos comparables.
    Postcondición: retorna una nueva lista ordenada; la original N
    """
    copia = datos[:]      # Copia superficial: trabajamos sobre la
                          # Ordenamiento por selección sobre la copia
    for i in range(len(copia) - 1):
        pos_min = i
        for j in range(i + 1, len(copia)):
            if copia[j] < copia[pos_min]:
                pos_min = j
        copia[i], copia[pos_min] = copia[pos_min], copia[i]
    return copia

# === Sección algorítmica ===
datos_originales = [38, 27, 43, 3, 9]
datos_ordenados = ordenar_sin_modificar(datos_originales)
print(datos_originales)  # [38, 27, 43, 3, 9] – intacta
print(datos_ordenados)   # [3, 9, 27, 38, 43]
```

Conexión con la Semana 6: sorted() vs. .sort()

La distinción que acabamos de estudiar es exactamente la que existe entre las dos formas de ordenar en Python: `sorted(lista)` devuelve una **nueva lista** ordenada (sin efecto secundario), mientras que `lista.sort()` ordena **in-place** y devuelve `None` (efecto secundario explícito). Ahora entendemos *por qué*: `sorted()` crea internamente una copia, y `.sort()` modifica el objeto original a través de la referencia.

7. Traza Completa: Referencias en Acción

Para consolidar los conceptos, tracemos paso a paso un programa que combina funciones, aliasing y mutabilidad. Seguimos la metodología de Pólya: primero analizamos, luego diseñamos mentalmente la traza, y finalmente verificamos.

```
def procesar(valores, umbral):
    """Elimina valores menores al umbral y devuelve el conteo de e

    Precondición: valores es una lista de números, umbral es un nú
    Postcondición: la lista queda sin los elementos menores a umbr
    Efecto secundario: modifica la lista recibida.
    """

    eliminados = 0
    i = 0
    while i < len(valores):
        if valores[i] < umbral:
            valores.pop(i)
            eliminados = eliminados + 1
        else:
            i = i + 1
    return eliminados

# === Sección algorítmica ===
datos = [3, 7, 1, 9, 2]
alias = datos # alias apunta al mismo objeto
cant = procesar(datos, 4)

print(datos)    # [7, 9]
print(alias)    # [7, 9] – también cambió (mismo objeto)
print(cant)     # 3
```

Paso	i	valores[i]	valores[i] < 4?	Acción	eliminados	Estado de la lista
------	---	------------	--------------------	--------	------------	--------------------------

Paso	i	valores[i]	valores[i] < 4?	Acción	eliminados	Estado de la lista
inicio	0	3			0	[3, 7, 1, 9, 2]
1	0	3	Sí	pop(0), no incrementa i	1	[7, 1, 9, 2]
2	0	7	No	i = 1	1	[7, 1, 9, 2]
3	1	1	Sí	pop(1), no incrementa i	2	[7, 9, 2]
4	1	9	No	i = 2	2	[7, 9, 2]
5	2	2	Sí	pop(2), no incrementa i	3	[7, 9]
6	2	—	i ≥ len(valores) ⇒ sale del while		3	[7, 9]

Observación clave sobre la traza

Nótese que cuando se elimina un elemento con `pop(i)`, **no se incrementa i**: el siguiente elemento «cae» a la posición recién liberada. Si incrementáramos `i` después de un `pop`, saltaríamos un elemento. Este patrón de recorrido con eliminación usando `while` (y no `for`) es la forma correcta cuando se modifica la longitud de la lista durante la iteración. Con `for`, el índice avanza automáticamente y se producen errores sutiles.

8. Conexión Holística: Gestión de Memoria en el Sistema Operativo

Hasta aquí hemos visto el modelo de memoria desde la perspectiva del programador Python. Ahora descendemos un nivel para entender cómo el **sistema operativo** implementa y gestiona ese modelo, conectando con los conceptos de arquitectura estudiados desde la Semana 1.

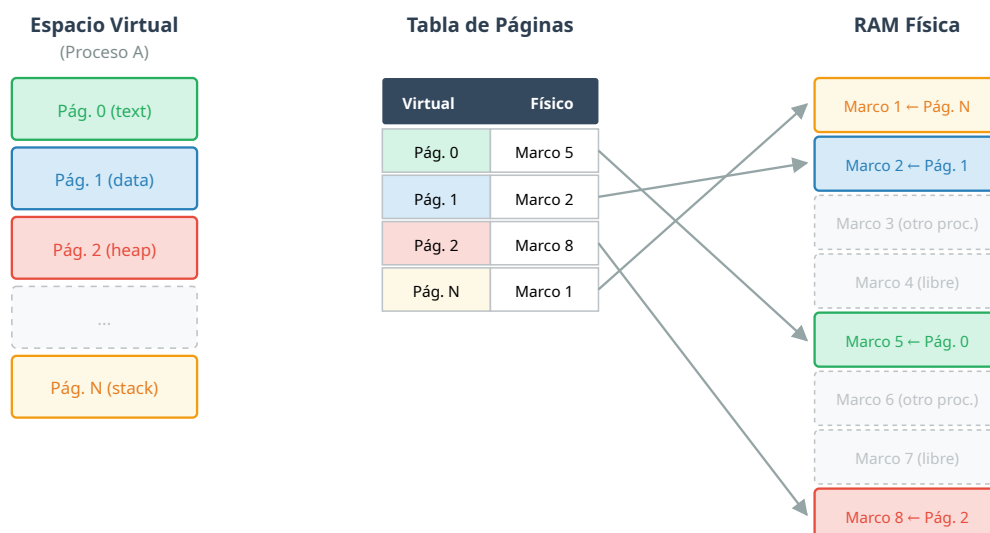
8.1. Memoria virtual: la ilusión del espacio propio

Cuando un programa se ejecuta, «cree» que tiene toda la memoria para sí. Esta ilusión la crea el sistema operativo mediante la **memoria virtual** : cada proceso trabaja con *direcciones virtuales* que el hardware traduce a *direcciones físicas* en la RAM real. Las ventajas de esta abstracción son fundamentales:

- **Aislamiento:** un proceso no puede acceder a la memoria de otro proceso, lo que proporciona seguridad y estabilidad.
- **Espacio uniforme:** cada proceso ve un espacio de direcciones contiguo que comienza en 0, independientemente de dónde estén físicamente los datos.
- **Sobrecompromiso:** el SO puede asignar más memoria virtual que la RAM física disponible, usando el disco como extensión (swap).

8.2. Paginación: dividir para gobernar

La memoria virtual se implementa mediante **paginación** : tanto el espacio virtual como la RAM física se dividen en bloques de tamaño fijo llamados **páginas** (típicamente de 4 KB). El sistema operativo mantiene una **tabla de páginas** por cada proceso que mapea páginas virtuales a marcos físicos (*frames*).



Observaciones clave sobre la paginación

No contiguo: las páginas de un proceso pueden estar dispersas en marcos físicos no contiguos. El programa «ve» memoria contigua, pero físicamente está fragmentada.

Compartición: dos procesos pueden mapear la misma página física (por ejemplo, el código de una biblioteca compartida), ahorrando memoria.

Page fault: si un proceso accede a una página que no está en RAM (porque fue enviada al disco), se produce un *fallo de página* y el SO la carga desde el disco —una operación miles de veces más lenta que un acceso a RAM.

8.3. Segmentación: división lógica

La **segmentación** es un esquema complementario donde la memoria se divide según la estructura lógica del programa: un segmento para el código, otro para los datos estáticos, otro para el heap y otro para el stack. Cada segmento tiene un tamaño variable (a diferencia de las páginas, que son de tamaño fijo).

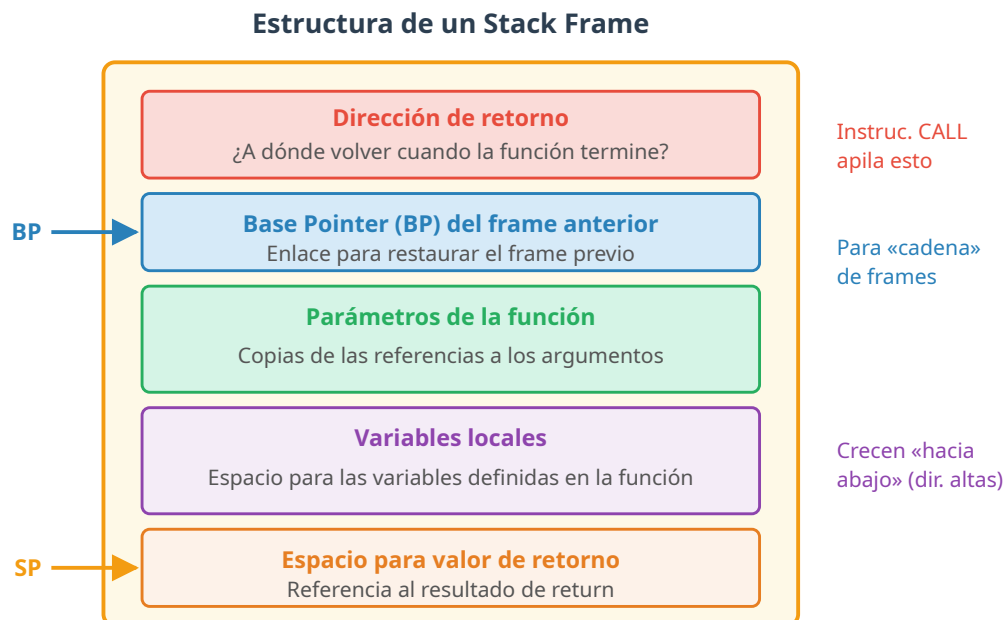
En la práctica, los sistemas operativos modernos (Linux, Windows, macOS) combinan ambos esquemas en lo que se denomina **segmentación paginada**: la división lógica en segmentos facilita la protección (el segmento text es de solo lectura, el stack tiene permisos de lectura/escritura pero no de ejecución), mientras que la paginación gestiona la colocación física en RAM.

Conexión con la jerarquía de memoria (Semana 5)

En la Semana 5, al estudiar la complejidad algorítmica, introducimos la **jerarquía de memoria** (registros → caché L1/L2 → RAM → disco) y el principio de **localidad de referencia**. La paginación es el mecanismo que conecta los dos últimos niveles de esa jerarquía: cuando un dato no está en RAM (page fault), el SO lo trae desde el disco. Y dentro de la RAM, el hardware de caché explota la localidad para mantener en L1/L2 las páginas más usadas recientemente.

8.4. La pila de ejecución en profundidad

En la Semana 4 introdujimos que cada invocación de función crea un *stack frame*. Ahora podemos comprender la estructura completa de un frame y cómo se gestiona a nivel del sistema:



Registros del CPU involucrados

SP (Stack Pointer): apunta siempre a la cima actual de la pila. Cuando se apila un dato, SP se decrementa (la pila crece hacia direcciones menores); cuando se desapila, SP se incrementa.

BP (Base Pointer): apunta a la base del frame actual. Permite acceder a parámetros y variables locales mediante desplazamientos fijos ($BP + \text{offset}$), incluso si SP cambia durante la ejecución de la función.

PC (Program Counter): contiene la dirección de la siguiente instrucción a ejecutar. La instrucción **CALL** apila el PC actual (la dirección de retorno) y carga la dirección de la función llamada. **RET** desapila la dirección y la carga en PC.

8.5. Ejemplo: pila de ejecución de un programa

Python

Veamos cómo se apilan y desapilan los frames en un programa concreto:

```
def es_par(n):  
    """Determina si n es par."""  
    return n % 2 == 0  
  
def contar_pares(lista):  
    """Cuenta cuántos elementos pares hay en la lista."""  
    cantidad = 0  
    for elem in lista:  
        if es_par(elem):           # Aquí se apila un frame para es_  
            cantidad = cantidad + 1  
    return cantidad  
  
# === Sección algorítmica ===  
datos = [3, 8, 5, 2]  
resultado = contar_pares(datos)  # Se apila frame de contar_pares  
print(resultado)                 # 2
```



Python Tutor: visualización interactiva

La herramienta **Python Tutor** (pythontutor.com) permite ejecutar código

Python paso a paso y observar los stack frames, las referencias a objetos en el heap y los efectos del aliasing. Es el complemento ideal para los diagramas que presentamos aquí: lo que dibujamos de forma estática, Python Tutor lo anima instrucción por instrucción. En la clase práctica de laboratorio utilizaremos esta herramienta extensivamente.

9. El Modelo de Memoria de Python: Síntesis

Con todos los conceptos desarrollados, podemos formular una síntesis precisa del modelo de memoria de Python que integra el nivel del lenguaje con el nivel del sistema operativo:

Modelo de memoria de Python: tres principios fundamentales

Principio 1 — Todo es un objeto: cada valor en Python (números, cadenas, listas, funciones, módulos) es un objeto que reside en el **heap**. Los objetos tienen tipo, valor e identidad (`id()`).

Principio 2 — Las variables son referencias: una variable es un nombre que reside en un **stack frame** y apunta a un objeto en el heap. La asignación (`=`) crea o reapunta una referencia; nunca copia el objeto. El paso de argumentos copia la referencia, no el objeto.

Principio 3 — La mutabilidad determina la visibilidad: si una función modifica un objeto mutable a través de su referencia, el cambio es visible desde cualquier otra referencia al mismo objeto. Si reasigna el parámetro, solo afecta al nombre local. Los objetos inmutables nunca pueden «parecer» modificados porque toda operación crea un objeto nuevo.

Concepto	Nivel del lenguaje (Python)	Nivel del SO / Hardware
Variable	Nombre que	Posición en el stack frame

Concepto	Nivel del lenguaje (Python)	Nivel del SO / Hardware
	referencia un objeto	que contiene una dirección
Objeto	Entidad con tipo, valor e id	Bloque de bytes en el heap
Asignación	<code>x = valor</code>	Escribir una dirección en la posición del stack
Aliasing	Dos variables con el mismo <code>id()</code>	Dos posiciones del stack con la misma dirección
Llamada a función	<code>f(argumento)</code>	Instrucción CALL: apilar PC, crear stack frame
Retorno	<code>return resultado</code>	Instrucción RET: desapilar frame, restaurar PC y BP
Memoria virtual	Espacio de direcciones del proceso	Tabla de páginas, traducción virtual → física
Desbordamiento de pila	<code>RecursionError</code>	Stack overflow: colisión stack-heap

10. Síntesis y Cierre

Esta semana hemos construido un modelo mental completo de cómo Python gestiona la memoria. Partimos del **modelo de cuatro segmentos** (text, data, heap, stack) que describe la organización del espacio de direcciones de cualquier proceso. Descubrimos que en Python **todo es un objeto** en el heap, y que las variables son **referencias** —nombres que apuntan a esos objetos—, almacenadas en los stack frames creados por cada invocación de función.

La distinción entre **mutabilidad e inmutabilidad** reveló por qué las listas se comportan de forma diferente a los enteros cuando se pasan como argumentos: no es que Python tenga dos mecanismos de paso, sino que tiene uno solo (paso

por asignación) cuyo efecto observable depende de si la función *modifica* el objeto (visible desde todos los alias) o *reasigna* el parámetro (solo afecta al nombre local).

En el plano holístico, la **memoria virtual** y la **paginación** nos mostraron cómo el SO crea la ilusión de un espacio de direcciones privado y contiguo para cada proceso, usando páginas de tamaño fijo que pueden estar dispersas en la RAM física. Y la estructura detallada del **stack frame** conectó los registros de la CPU (SP, BP, PC) con las instrucciones CALL y RET que hacen posible la invocación y el retorno de funciones.

Conexiones del tema



Semanas 1–6: La arquitectura Von Neumann (Semana 1), las funciones y el scope (Semana 4), la jerarquía de memoria (Semana 5) y los algoritmos de ordenamiento in-place (Semana 6) son los cimientos sobre los que construimos el modelo de memoria. Ahora entendemos por qué el swap funciona en las funciones de ordenamiento: porque la lista se pasa por referencia y se modifica in-place.



Semana 7 (este tema): Modelo de memoria (text, data, heap, stack). Referencias, identidad (`id()`), `is` vs. `==`. Mutabilidad e inmutabilidad. Aliasing y copias. Paso de argumentos por asignación. Efectos secundarios. Gestión de memoria en el SO: memoria virtual, paginación, segmentación. Stack frames en detalle.



Semana 8: Archivos binarios y el módulo `struct`. La comprensión de la representación binaria en memoria (cómo se almacenan `int`, `float`, `str` como secuencias de bytes) es esencial para diseñar registros de longitud fija. La noción de *endianness* conecta la representación en memoria con la representación en disco. La analogía entre la *free list* del archivo y la *free list* del heap retoma directamente los conceptos de esta semana.



Semanas posteriores: La **recursividad** (Semana 9) se

comprende como apilamiento sucesivo de frames —y el `RecursionError` como agotamiento del stack. El **garbage collector** (Semana 9) gestiona la liberación automática de objetos del heap. Las **estructuras de datos** (Semana 10) exploran cómo listas, diccionarios y conjuntos se representan internamente. El **proyecto final** (Semanas 13–16) exigirá diseñar funciones con contratos claros sobre efectos secundarios y manejo eficiente de memoria.

Conexión holística: memoria, hardware y sistema operativo

El modelo de memoria que hemos estudiado cruza todos los niveles de la pila computacional. En el nivel más bajo, los **registros de la CPU** (SP, BP, PC) gestionan mecánicamente los stack frames mediante las instrucciones CALL y RET del set de instrucciones. En el nivel intermedio, el **sistema operativo** crea la ilusión de memoria contigua mediante la paginación, protege los segmentos de cada proceso y gestiona los fallos de página cuando la RAM se agota. En el nivel más alto, el **intérprete Python** (CPython) implementa su propio allocator de memoria sobre el heap del SO, con un *garbage collector* que libera automáticamente los objetos que ya no tienen referencias. Cada nivel de abstracción simplifica la vista del nivel superior, pero la comprensión de todos los niveles es lo que distingue a un programador que *sabe lo que hace* de uno que *solo sabe cómo hacerlo*.